

Aula 12 - Documentação de Software

Duração: 2 horas

Objetivos:

- Dominar técnicas de documentação eficiente em Java
- Gerar documentação automatizada para APIs REST
- Adotar práticas ágeis de documentação

Parte Teórica (40 min)

1. Hierarquia da Documentação

Nível	Ferramentas	Público-Alvo
Código	Javadoc, Anotações	Devs
API	Swagger/OpenAPI	Devs Frontend/Integração
Sistema	Markdown, Confluence	POs, Stakeholders

2. Princípios da Boa Documentação

- ✓ **Viva:** Atualizada automaticamente quando possível
- ✓ **Busqueável:** Estrutura clara e indexável
- ✓ **Relevante:** Evite documentação óbvia

Parte Prática (1h20min)

Atividade 1: Javadoc Avançado

1. Documente uma classe com:

- Descrição da classe
- `@author` , `@version`
- `@throws` , `@param` , `@return`

java

```
/**
 * Calculadora de operações financeiras básicas.
 * @author Maria Silva
 * @version 1.1
 */
public class Financeira {

    /**
     * Calcula juros compostos.
     * @param principal Valor inicial
     * @param taxa Taxa mensal (0.1 = 10%)
     * @param meses Período
     * @return Valor futuro
     * @throws IllegalArgumentException Se taxa ou meses forem negativos
     */
    public double jurosCompostos(double principal, double taxa, int meses) {
        if (taxa < 0 || meses < 0) {
            throw new IllegalArgumentException("Parâmetros não podem ser negativos");
        }
        return principal * Math.pow(1 + taxa, meses);
    }
}
```

2. Gerar HTML:

bash

```
mvn javadoc:javadoc
# Acesse: target/site/apidocs/index.html
```

Exercício:

- Documente uma classe do seu projeto com:
 - Exemplo de uso (`@code`)
 - Link para método relacionado (`@see`)

Atividade 2: Swagger/OpenAPI

1. Adicione ao `pom.xml` :

xml

```
<dependency>
    <groupId>org.springdoc</groupId>
```

```
<artifactId>springdoc-openapi-ui</artifactId>
<version>1.7.0</version>
</dependency>
```

2. Anote seu controller:

java

```
@Operation(summary = "Obter usuário por ID")
@ApiResponses(value = {
    @ApiResponse(responseCode = "200", description = "Usuário encontrado"),
    @ApiResponse(responseCode = "404", description = "Usuário não encontrado")
})
@GetMapping("/usuarios/{id}")
public Usuario getUsuario(@Parameter(description = "ID do usuário") @PathVariable Long id) {
    return service.findById(id);
}
```

3. Acesse a UI:

<http://localhost:8080/swagger-ui.html>

Exercício:

- Documente um endpoint POST com schema do requestBody

Atividade 3: Documentação Ágil

1. Exemplo de README.md:

markdown

```
# Sistema de Pagamentos
```

```
## Requisitos
```

- Java 17+
- PostgreSQL 14

```
## Como Executar
```

```
```bash
mvn spring-boot:run
```
```

```
## Rotas Principais
```

| Endpoint | Método | Descrição |
|----------|--------|-----------|
|----------|--------|-----------|

```
|-----|-----|-----|
| `/pagamentos` | POST | Cria novo pagamento|
```

2. Template para Confluence:

markdown

h2. Visão Geral

Data de Criação: {{data}}

Responsável: {{owner}}

h3. Fluxo Principal

{code:java}

```
public void processar() { ... }
```

{code}

h3. Decision Records

||Data||Decisão||Razão||

|12/05|Usar JWT|Mais stateless|

Exercício:

- Crie um arquivo `ARCHITECTURE.md` com:
 - Diagrama de componentes (usando Mermaid)
 - Principais decisões técnicas

Desafio Integrado (30 min)

Contexto:

1. Em grupos de 3:

- **Dev 1:** Documente 1 classe via Javadoc
- **Dev 2:** Documente 1 endpoint REST
- **Dev 3:** Atualize o README.md

2. Critérios:

- Javadoc deve gerar HTML sem warnings
- Swagger UI deve mostrar exemplos
- README deve permitir execução local

Encerramento

- **Discussão:** Qual tipo de documentação traz mais valor?
- **Próxima aula:** Preparação para o Projeto Final

Tarefa:

- Documentar 2 endpoints do projeto final com OpenAPI
- Gerar site de documentação (`mvn site`)

Recursos:

- [Javadoc Guide](#)
- [OpenAPI Specification](#)

Documentação é código que fala! Dúvidas? 😊📖